

Annotations for Reflection

Document #: P3394R0 [[Latest](#)] [[Status](#)]
Date: 2024-10-14
Project: Programming Language C++
Audience: EWG
Reply-to: Wyatt Childers
<wcc@edg.com>
Dan Katz
<dkatz85@bloomberg.net>
Barry Revzin
<barry.revzin@gmail.com>
Daveed Vandevoorde
<daveed@edg.com>

Contents

[1 Introduction](#)

[2 Motivating Examples](#)

[2.1 Command-Line Argument Parsing](#)

[2.2 Test Parametrization](#)

[2.3 Serialization](#)

[3 Proposal](#)

[3.1 Why not Attributes?](#)

[3.2 Library Queries](#)

[3.3 Additional Syntactic Constraints](#)

[3.4 Implementation Experience](#)

[3.5 Other Directions We Are Exploring](#)

[4 References](#)

§ 1 Introduction

Ever since writing [[P1240R0](#)] (“Scalable Reflection in C++”), but more so since [[P2996R0](#)] (“Reflection for C++26”), we have been requested to add a capability to annotate declarations in a way that reflection can observe. For example, Jeremy Ong presented compelling arguments in a post to the SG7 reflector: <https://lists.isocpp.org/sg7/2023/10/0450.php>. Corentin Jabot also noticed the need while P1240 was evolving and wrote [[P1887R0](#)] (“Reflection on Attributes”), which proposes syntax not entirely unlike what we present here.

In early versions of P2996 (and P1240 before that), a workaround was to encode properties in the template arguments of alias template specializations:

```
template <typename T, auto... Annotations>
using Noted = T;

struct C {
    Noted<int, 1> a;
    Noted<int*, some, thing> b;
};
```

It was expected that something like `type_of(^C::a)` would produce a reflection of `Noted<int, 1>` and that can be taken apart with metafunctions like `template_arguments_of` — which both preserves the type as desired (a is still an `int`) and allows reflection queries to get at the desired annotations (the `1`, `some`, and `thing` in this case).

There are problems with this approach, unfortunately:

- It doesn't work for all contexts where we might want to annotate declarations (e.g., enumerators).
- It doesn't directly express the intent.
- It turns out that providing access to aliases used in the declaration of reflected entities raises difficult questions and P2996 is therefore likely going to drop the ability to access that information (with the intention of resurrecting the capability with a later paper once specification and implementation challenges have been addressed).

In this paper, we propose simple mechanisms that more directly support the ability to annotate C++ constructs.

§ 2 Motivating Examples

We'll start with a few motivating examples for the feature. We'll describe the details of the feature in the subsequent section.

These examples are inspired from libraries in other programming languages that provide some mechanism to annotate declarations.

§ 2.1 Command-Line Argument Parsing

Rust's [clap](#) library provides a way to add annotations to declarations to help drive how the parser is declared. We can now [do the same](#):

```
struct Args {
    [[=clap::Help("Name of the person to greet")]]
    [[=clap::Short, =clap::Long]]
    std::string name;

    [[=clap::Help("Number of times to greet")]]
    [[=clap::Short, =clap::Long]]
    int count = 1;
```

```
};

int main(int argc, char** argv) {
    Args args = clap::parse<Args>(argc, argv);

    for (int i = 0; i < args.count; ++i) {
        std::cout << "Hello " << args.name << '\n';
    }
}
```

Here, we provide three types (Short, Long, and Help) which help define how these member variables are intended to be used on the command-line. This is implemented on top of [Lyra](#).

When run:

```
$ demo -h
USAGE:
  demo [-?|-h|--help] [-n|--name <name>] [-c|--count <count>]

Display usage information.

OPTIONS, ARGUMENTS:
  -?, -h, --help
  -n, --name <name>      Name of the person to greet
  -c, --count <count>    Number of times to greet

$ demo -n wg21 --count 3
Hello wg21
Hello wg21
Hello wg21
```

While Short and Long can take explicit values, by default they use the first letter and whole name of the member that they annotate.

The core of the implementation is that `parse<Args>` loops over all the non-static data members of `Args`, then finds all the `clap`-related annotations and invokes them:

```
template <typename Args>
auto parse(int argc, char** argv) -> Args {
    Args args;
    auto cli = lyra::cli();

    // ...

    template for (constexpr info M : nonstatic_data_members_of(^Args))
    {
        auto id = std::string id(identifier_of(mem));
        auto opt = lyra::opt(args.[:M:], id);

        template for (constexpr info A : annotations_of(M)) {
```

```

        if constexpr (parent_of(type_of(A)) == ^^clap) {
            // for those annotations that are in the clap namespace
            // invoke them on our option
            extract<[:type_of(A):]>(A).apply_annotation(opt, id);
        }
    }

    cli.add_argument(opt);
}

// ...
};

```

So, for instance, Short would be implemented like this:

```

namespace clap {
    struct ShortArg {
        // optional isn't structural yet but let's pretend
        optional<char> value;

        constexpr auto operator()(char c) const -> ShortArg {
            return {.value=c};
        };

        auto apply_annotation(lyra::opt& opt, std::string_view id) const
        -> void {
            char first = value.value_or(id[0]);
            opt[std::string("-") + first];
        }
    };

    inline constexpr auto Short = ShortArg();
}

```

Overall, a fairly concise implementation for an extremely user-friendly approach to command-line argument parsing.

§ 2.2 Test Parametrization

The pytest framework comes with a decorator to [parametrize](#) test functions. We can now do [the same](#):

```

namespace N {
    [[parametrize(
        Tuple{1, 1, 2},
        Tuple{1, 2, 3}
    )]]
    void test_sum(int x, int y, int z) {
        std::println("Called test_sum(x={}, y={}, z={})", x, y, z);
    }

    struct Fixture {

```

```

    Fixture() {
        std::println("setup fixture");
    }

    ~Fixture() {
        std::println("teardown fixture");
    }

    [[=parametrize({Tuple{1}, Tuple{2}})]]
    void test_one(int x) {
        std::println("test one({})", x);
    }

    void test_two() {
        std::println("test two");
    }
};

int main() {
    invoke_all<^^N>();
}

```

When run, this prints:

```

Called test_sum(x=1, y=1, z=2)
Called test_sum(x=1, y=2, z=3)
setup fixture
test one(1)
teardown fixture
setup fixture
test one(2)
teardown fixture
setup fixture
test two
teardown fixture

```

Here, `parametrize` returns a value that is some specialization of `Parametrize` (which is basically an array of tuples, except that `std::tuple` isn't structural so the implementation rolls its own).

The rest of the implementation looks for all the free functions named `test_*` or nonstatic member functions of class types that start with `test_*` and invokes them once, or with each parameter, depending on the presence of the annotation. That looks like this:

```

constexpr auto parametrization_of(std::meta::info M) -> std::meta::info
{
    for (auto a : annotations_of(M)) {
        auto t = type_of(a);
        if (has_template_arguments(t) and template_of(t) ==
            ^^Parametrize) {
            return a;
        }
    }
}

```

```

    }
}
return std::meta::info();
}

template <std::meta::info M, class F>
void invoke_single_test(F f) {
    constexpr auto A = parametrization_of(M);

    if constexpr (A != std::meta::info()) {
        // if we are parametrized, pull out that value
        // and for each tuple, invoke the function
        // this is basically calling std::apply on an array of tuples
        constexpr auto Params = extract<[:type_of(A):]>(A);
        for (auto P : Params) {
            P.apply(f);
        }
    } else {
        f();
    }
}

template <std::meta::info Namespace>
void invoke_all() {
    template for (constexpr std::meta::info M : members_of(Namespace)) {
        if constexpr (is_function(M) and
            identifier_of(M).starts_with("test_")) {
            invoke_single_test<M>([:M:]);
        } else if constexpr (is_type(M)) {
            template for (constexpr std::meta::info F :
                nonstatic_member_functions_of(M)) {
                if constexpr (identifier_of(F).starts_with("test_")) {
                    invoke_single_test<F>([&](auto... args){
                        typename [:M:] fixture;
                        fixture.[:F:] (args...);
                    });
                }
            }
        }
    }
}

```

§ 2.3 Serialization

Rust's [serde](#) library is a framework for serialization and deserialization. It is easy enough with reflection to do member-wise serialization. But how do you opt into that? An annotation provides a cheap mechanism of [doing just that](#) (built on top of [Boost.Json](#)):

```

struct [[=serde::derive]] Point {
    int x, y;
};

```

Allowing:

```
// prints {"x":1,"y":2}
std::cout << boost::json::value_from(Point{.x=1, .y=2});
```

But opting in is just the first thing you might want to do with serialization. You might also, for instance, want to change how fields are serialized. `serde` provides a lot of attributes to do so. The easiest to look at is `rename`, which uses the provided string instead of the name of the non-static data member:

```
struct [[serde::derive]] Person {
    [[serde::rename("first name")]] std::string first;
    [[serde::rename("last name")]] std::string last;
};
```

Which leads to:

```
// prints {"first name":"Peter","last name":"Dimov"}
std::cout << boost::json::value_from(Person{.first="Peter",
    .last="Dimov"});
```

The implementation for these pieces is fairly straightforward. We provide an opt-in for the value conversion function when the `serde::derive` annotation is present. In that case, we walk all the non-static data members and write them into the `boost::json::value` output. If a `serde::Rename` annotation is present, we use that instead of the data member's name:

```
namespace serde {
    inline constexpr struct{} derive{};
    struct rename { char const* field; };
}

namespace boost::json {
    template <class T>
        requires (has_annotation(^T, serde::derive))
    void tag_invoke(value_from_tag const&, value& v, T const& t) {
        auto& obj = v.emplace_object();
        template for (auto M : nonstatic_data_members_of(^T)) {
            constexpr auto field = annotation_of<serde::rename>(M)
                .transform([](serde::rename r){
                    return std::string_view(r.field);
                })
                .value_or(identifier_of(M));

            obj[field] = boost::json::value_from(t.[:M:]);
        }
    }
}
```

You can imagine extending this out to support a wide variety of other serialization-specific attributes that shouldn't otherwise affect the C++ usage of the type. For instance, a [more complex approach](#) additionally supports the `skip_serializing_if` annotation while first collecting all `serde` annotations into a struct.

§ 3 Proposal

The core idea is that an *annotation* is a compile-time value that can be associated with a construct to which attributes can appertain. Annotation and attributes are somewhat related ideas, and we therefore propose a syntax for the former that builds on the existing syntax for the latter.

At its simplest:

```
struct C {  
    [[=1]] int a;  
};
```

Syntactically, an annotation is an attribute of the form `= expr` where `expr` is a *constant-expression* (which syntactically excludes, e.g., *comma-expression*) to which the glvalue-to-prvalue conversion has been applied if the expression wasn't a prvalue to start with.

Currently, we require that an annotation has structural type because we're going to return annotations through `std::meta::info`, and currently all reflection values must be structural.

§ 3.1 Why not Attributes?

Attributes are very close in spirit to annotations. So it made sense to piggy-back on the attribute syntax to add annotations. Existing attributes are designed with fairly open grammar and they can be ignored by implementations, which makes it difficult to connect them to user code. Given a declarations like:

```
[[nodiscard, gnu::always_inline]]  
[[deprecated("don't use me")]]  
void f();
```

What could reflecting on `f` return? Because attributes are ignorable, an implementation might simply ignore them. Additionally, there is no particular value associated with any of these attributes that would be sensible to return. We're limited to returning either a sequence of strings. Or, with [P3294](#), token sequences.

But it turns out to be quite helpful to preserve the actual values without requiring libraries to do additional parsing work. Thus, we need to distinguish annotations (whose values we need to preserve and return back to the user) from attributes (whose values we do not). Thus, we looked for a sigil introducing a general expression.

Originally, the plus sign (+) was considered (as in P1887), but it is not ideal because a prefix + has a meaning for some expressions and not for others, and that would not carry over to the attribute notation.

A prefix `=` was found to be reasonably meaningful in the sense that the annotation “equals” the value on the right, while also being syntactically unambiguous. We also discussed using the reflection operator (`^`) as an introducer (which is attractive because the annotation ultimately comes back to the programmer as a reflection value), but that raised questions about an annotation itself being a reflection value (which is not entirely improbable).

As such, this paper proposes annotations as distinct from attributes, introduced with a prefix `=`.

§ 3.2 Library Queries

We propose the following set of library functions to work with annotations:

```
namespace std::meta {
    consteval bool is_annotation(info);

    consteval vector<info> annotations_of(info item);           // (1)
    consteval vector<info> annotations_of(info item, info type); // (2)
    template<class T>
        consteval optional<T> annotation_of(info item);

    template<class T>
        consteval bool has_annotation(info item);             // (3)
    template<class T>
        consteval bool has_annotation(info item, T const& value); // (4)

    consteval info annotate(info item,
                           info value,
                           source_location loc =
                               source_location::current());
}
```

`is_annotation` checks whether a particular reflection represents an annotation.

We provide two overloads of `annotations_of` to retrieve all the annotations on a particular item:

1. Returns all the annotations.
2. Returns all the annotations `a` such that `type_of(a) == type`.

And a singular version, `annotation_of<T>`, that returns the annotation `a` such that `dealias(type_of(a)) == ^T`. If no such annotation exists, returns `nullopt`. If more than one such annotation exists and all the values are not template-argument-equivalent, this call is not a constant expression.

And then two overloads of `has_annotation` that simply checks if a given annotation exists:

3. Checks if there exists an annotation `a` such that `dealias(type_of(a)) == ^T`.
4. Checks if there exists an annotation `a` such that `value_of(a) == reflect_value(value)`.

Of these, four can be directly implemented in terms of the unary `annotations_of(item)`, but we think they'll be common enough to merit inclusion.

And lastly, `annotate` provides the ability to programmatically add an annotation to a declaration.

§ 3.3 Additional Syntactic Constraints

Annotations can be repeated:

```
[[=42, =42]] int x;  
static_assert(annotations_of(^x).size() == 2);
```

Annotations spread over multiple declarations of the same entity accumulate:

```
[[=42]] int f();  
[[=24]] int f();  
static_assert(annotations_of(^f).size() == 2);
```

Annotations follow appertenance rules like attributes, but shall not appear in the *attribute-specifier-seq* of a *type-specifier-seq* or an *empty-declaration*:

```
struct [[=0]] S {}; // Okay: Appertains to S.  
[[=42]] int f();    // Okay: Appertains to f().  
int f[[=0]] f();    // Ditto.  
int [[=24]] f();    // Error: Cannot appertain to int.  
[[=123]]            // Error: No applicable construct.
```

To avoid confusion, annotations are not permitted after an *attribute-using-prefix*. So this is an error:

```
[[using clang: amdgpu_waves_per_eu, =nick("weapon")]]  
int select_footgun(int);
```

Instead, use:

```
[[using clang: amdgpu_waves_per_eu]] [[=nick("weapon")]]  
int select_footgun(int);
```

§ 3.4 Implementation Experience

The core language feature and the basic query functions have been implemented in the EDG front end and in Bloomberg's P2996 Clang fork (with option `-freflection-latest`), both available on Compiler Explorer.

§ 3.5 Other Directions We Are Exploring

As evidenced in the motivating examples earlier, there is a lot of value in this proposal even in this simple form. However, there is more to consider when it comes to annotations.

This proposal right now lets us unconditionally add an annotation to a type:

```
struct [[=X]] Always;
```

But it does not let us conditionally add an annotation to a type:

```
template <class T>
struct /* X only for some T */ Sometimes;
```

Or to really generalize annotations. For instance, in the clap example earlier, our example showed usage with `clap::Short` and `clap::Long`. What if somebody wants to compose these into their own annotation that attaches both `clap::Short` and `clap::Long` to a declaration?

More broadly, there is clear value in having an annotation be able to be invoked by the declaration itself. Doing so allows the two uses above easily enough. An interesting question, though, is whether this callback (syntax to be determined) is invoked at the *beginning* of the declaration or at the *end* of the declaration. For annotations on classes, this would be before the class is complete or after the class is complete. Before completeness allows the class to observe the annotation during instantiation. After completeness allows the annotation callback to observe properties of the type. In some sense, Herb Sutter's [P0707R4] ("Metaclass functions: Generative C++") was adding annotations on classes, invoked on class completeness, that allow mutation of the class.

One concrete, simpler example. We can, with this proposal as-is, create a `Debug` annotation that a user can add to their type and a specialization of `std::formatter` for all types that have a `Debug` annotation [as follows](#):

```
template <auto V> struct Derive { };
template <auto V> inline constexpr Derive<V> derive;

inline constexpr struct{} Debug;

template <class T> requires (has_annotation(^T, derive<Debug>))
struct std::formatter<T> {
    // ...
};

struct [[=derive<Debug>]] Point {
    int x;
    int y;
};

int main() {
```

```
auto p = Point{.x=1, .y=2};  
// prints p=Point{.x=1, .y=2}  
std::println("p={}", p);  
}
```

This *works*, but it's not really the ideal way of doing it. This could still run into potential issues with ambiguous specialization of `std::formatter`. Better would be to allow the `Debug` annotation to, at the point of completion of `Point`, inject an explicit specialization of `std::formatter`. This would rely both on the ability for the annotation to be called back and language support for such injection (see [P3294R1] (“Code Injection with Token Sequences”)).

There are still open questions as to how to handle such callbacks. Does an annotation that gets called back merit different syntax from an annotation that doesn't? Can it mutate the entity that it is attached to? How do we name the potential callbacks? Should the callback be registered implicitly (e.g., if an annotation of type `X` with member `X::annotate_declaration(...)` appears, that member is automatically a callback invoked when an entity is first declared with an annotation of type `X`) or explicitly (e.g., calling `annotated_declaration_callback(^^X, X_handler)` would cause `X_handler(...)` to be invoked when an entity is first declared with an annotation of type `X`).

§ 4 References

[P0707R4] Herb Sutter. 2019-06-17. Metaclasses: Generative C++.

<https://wg21.link/p0707r4>

[P1240R0] Andrew Sutton, Faisal Vali, Daveed Vandevoorde. 2018-10-08. Scalable Reflection in C++.

<https://wg21.link/p1240r0>

[P1887R0] Corentin Jabot. 2019-10-06. Typesafe Reflection on attributes.

<https://wg21.link/p1887r0>

[P2996R0] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde. 2023-10-15. Reflection for C++26.

<https://wg21.link/p2996r0>

[P3294R1] Barry Revzin, Andrei Alexandrescu, Daveed Vandevoorde. 2024-07-16. Code Injection with Token Sequences.

<https://wg21.link/p3294r1>